

Escuela de
Ingeniería y Arquitectura
Universidad Zaragoza

Universidad de Zaragoza

Facultad de Ingeniería

Artificial Intelligence - Lab Session 2: RL and GPs

Jorge Ruiz González (826685)
Hugo López Sola (875455)

October 4th, 2026

Contents

1 Introduction **2**

2 Questions 1–7 && 9 **2**

2.1 Question 1: Value Iteration 2
2.2 Question 2: Bridge Crossing Analysis 2
2.3 Question 3: Policies 2
2.4 Question 4: Q-learning 3
2.5 Question 5: Epsilon Greedy 3
2.6 Question 6: Bridge Crossing Revisited 3
2.7 Question 7: Q-Learning and Pacman 3
2.8 Question 9: Policy search 3

1 Introduction

In this lab session we applied the concepts studied in class for Reinforcement Learning. We firstly implemented value iteration and Q-learning, testing the agents on different problems: GridWorld, PacMan, Crawler Robot, etc.

The project includes an autograder to analyze if the behaviour of the agents suites what is expected for them.

2 Questions 1–7 && 9

2.1 Question 1: Value Iteration

Given a non terminal state, we compute the Q-value for each action we can take. We can use that value to estimate the total reward we will obtain. We define a policy by taking always the action with highest Q-value.

$$Q_{k+1}(x,a) = \sum_{x'} p(x' \mid x,a)\big[R(x,a,x') + \gamma V_k(x')\big].$$

The best action with this policy is $\pi_{k+1}(x) = \arg\max_a Q_{k+1}(x,a)$, and the next value estimate is $V_{k+1}(x) = \max_a Q_{k+1}(x,a)$. Each iteration writes to a new value table, so every update uses the same previous estimates (batch value iteration). Terminal states have value zero and no action. This computes a policy before interacting with the environment.

2.2 Question 2: Bridge Crossing Analysis

We keep the discount at $\gamma = 0.9$ and reduce transition noise from 0.2 to 0.01. With lower noise value, the agent interprets that its safer to cross the bridge, as there is lower risk of making unintended movements. Then, with this combination of parameters the future reward outweighs the risk of falling.

2.3 Question 3: Policies

The discount γ controls the importance of future rewards, noise determines the risk of making unintended movements, and the living reward rewards or penalizes each non-exit step.

Part	Desired behaviour	γ	Noise	Living reward
3(a)	Close exit, near cliff	0.20	0.001	−0.20
3(b)	Close exit, avoid cliff	0.20	0.100	+0.10
3(c)	Distant exit, near cliff	0.95	0.001	−0.20
3(d)	Distant exit, avoid cliff	0.95	0.100	+0.20
3(e)	Avoid exits and cliff	0.01	0.200	−0.20

In **3(a)** and **3(b)**, low discount favours the close exit. In 3(a), low noise and a negative living reward favour the short route near the cliff. In 3(b), higher noise and a positive living reward favour the longer safe route.

In **3(c)** and **3(d)**, high discount favours the distant +10 exit. In 3(c), low noise and a negative living reward favour the short route. In 3(d), higher noise and a positive living reward favour the longer safe route.

In **3(e)**, very low discount makes distant exit rewards almost irrelevant. Together with transition risk, this produces a cycle between $(0,1)$ and $(0,2)$ on the required noiseless path. The negative living reward does not encourage survival, and actual noise could eventually break the cycle.

2.4 Question 4: Q-learning

Value iteration uses the known MDP model to compute a policy before interacting with the environment. However, Q-learning agents learn from experience, updating the Q-values after each observed transition without requiring the transition or reward model.

In `computeValueFromQValues`, we evaluate all legal actions using `getQValue` and return the highest value.

In `computeActionFromQValues`, we collect all actions with that maximum value and use `random.choice` to break ties. With no legal actions, these methods return zero and `None`, respectively.

The method `update` modifies only the Q-value of the observed state–action pair:

$$Q(x_t, a_t) \leftarrow (1-\alpha)Q(x_t, a_t) + \alpha\big[R_{t+1} + \gamma V(x_{t+1})\big],$$

where $V(x_{t+1})$ is obtained using `computeValueFromQValues`.

2.5 Question 5: Epsilon Greedy

With probability ϵ, the agent will now take a random legal action uniformly, instead of always taking the best possible action (highest Q-value). Small ϵ values take more advantage of what the agent learnt, and large ϵ values make the agent explore more but also take bad actions.

For the crawler problem, a good strategy is to start with moderate exploration ($\epsilon \approx 0.5$) and a moderate learning rate. Then, we could gradually reduce both when the crawler has learned how to make efficient movements. This benefits exploration and leads to better movement patterns later. Small updates make the Q-values stabilize.

2.6 Question 6: Bridge Crossing Revisited

With $\epsilon = 1$, the agent behaves completely randomly and does not learn a reliable route across the bridge. With $\epsilon = 0$, the agent only exploits its current knowledge and may never explore the bridge sufficiently, as it keeps next to the starting point. Intermediate epsilon values provide exploration, but random actions can still make the agent fall from the bridge. The learning rate controls how quickly new experiences replace previous Q-values, but it cannot remove this exploration problem. After only 50 episodes, the results are too dependent on random actions and initialisation. Therefore, no fixed combination of epsilon and learning rate is more than 99% likely to learn the optimal policy.

2.7 Question 7: Q-Learning and Pacman

Approximate Q-learning represents each state-action value using a weighted combination of features instead of storing every pair independently. During training, the agent updates the feature weights according to the difference between the predicted and target Q-values. This allows the knowledge learnt in one state to generalise to other similar states. After training for 2000 games, the agent was evaluated in 100 additional games on smallGrid. The agent achieved 100 wins, so it passed the required threshold of 70 wins.

2.8 Question 9: Policy search

This experiment applies the REINFORCE algorithm to control the CartPole environment. The agent uses a linear policy followed by a softmax function to select actions. During training, actions are sampled from the policy to encourage exploration. The loss is based on the return-to-go obtained after each action. Adam updates the policy parameters using the calculated policy gradient. The experiment uses a learning rate of 0.01 and a discount factor of 1.0. Training is performed for 1000 episodes, with evaluation every 100 episodes, as in the given code example.

The results can vary, as the initial states and sampled actions are stochastic, but the current model has been capable of keeping the bar in equilibrium for more than 10 seconds. Overall, the experiment shows how policy gradients can learn to keep the pole upright.